Explicit Semantic Categories for More Legible Mainstream Programming Languages

Jayanta Majumder, Sambuddha Majumder

Abstract

Mainstream object-oriented languages have historically assigned an unusually large number of semantic roles to a small set of linguistic constructs, especially the `class`. In large C++ and Java codebases, a class may denote a passive compound data record, a stateful encapsulated abstraction, a namespace-like module, a service facade, a callback object, a function closure surrogate, a policy object, or a nominal wrapper whose purpose is only to encode an additional static property. Although this uniformity can reduce the number of language constructs, it also suppresses information that is valuable to readers, static-analysis tools, code browsers, and maintainers. A developer encountering a large class hierarchy must often recover the intended architectural role of each declaration indirectly from naming conventions, member patterns, documentation, or implementation details.

This paper argues for a modest alternative: mainstream languages should expose a small number of *semantic declaration categories* corresponding to materially different programming roles. Records should primarily express data composition; modules should express grouping and information hiding without object identity; stateful objects should express encapsulated mutable abstractions; closures or callable declarations should express behaviour as values; and statically tracked properties should be represented separately from nominal data types. The proposal is not to abolish classes, nor to maximize syntactic variety, but to align syntax with distinctions that are already present in software architecture. Such distinctions can improve codebase legibility while also enabling the compiler to enforce role-specific restrictions. The result is a language design philosophy in which syntax carries architectural intent rather than requiring that intent to be inferred from convention.

1 Introduction

One of the most successful ideas in modern programming-language design is abstraction by means of user-defined types. In C++ and Java, the class became the central mechanism through which programmers could group data, attach operations, hide representation, exploit subtype polymorphism, and organize large systems. Its generality is a major reason for its success.

Generality, however, has a cost. In sufficiently large systems, the same syntactic category is routinely used for declarations with sharply different meanings. A class with ten public fields and no meaningful invariants is functioning as a record. A class containing only static methods is functioning as a module or namespace. A one-method class carrying captured state may function as a callback or closure. A small wrapper around a string may exist only to encode a validation fact. A stateful object coordinating access to an external resource has a substantially different semantic role from all of these, even though each may be introduced with the same keyword and occupy the same nominal type namespace.

This paper examines whether those distinctions deserve direct linguistic expression. The central thesis is:

> When program entities have materially different semantic roles, and those roles matter to human understanding or static enforcement, a programming language

benefits from making at least some of those distinctions explicit in its syntax and static semantics.

The aim is deliberately pragmatic. The proposal does not require dependent types, proof assistants, macro metaprogramming, or a wholesale rejection of object-oriented programming. It instead suggests that a mainstream compiled language can remain familiar while replacing a single semantically overloaded declaration category with a small family of purpose-revealing constructs.

2 The Semantic Overloading of `class`

In conventional object-oriented languages, the class often serves several roles that are conceptually independent.

2.1 Compound data records

A class may primarily group fields into a compound value:

```
class Person {
    String name;
    int age;
}
```

Although methods may be present, the principal abstraction is structural: a `Person` is a product of named components. Java's introduction of records is itself evidence that this role is sufficiently distinct to justify dedicated syntax. Records make a specific data-oriented intention visible and permit the language to supply behaviour appropriate to that intention.

2.2 Stateful encapsulated objects

A second role is the classical object abstraction: state is hidden behind an interface, and the implementation protects invariants across a sequence of operations.

```
object BankAccount {
    private Money balance;

    public void deposit(Money m) { ... }
    public void withdraw(Money m) { ... }
}
```

Here identity, lifetime, mutation, and representation hiding are central. This entity is fundamentally different from a passive record even if both are representable as classes.

2.3 Modules and service namespaces

A class may also be used merely to group operations:

```
final class TaxRules {
    private TaxRules() {}

    static Money calculateTax(Income x) { ... }
    static Rate marginalRate(Income x) { ... }
}
```

The constructor is suppressed because no instances are conceptually required. The class is serving as a namespace or module, yet this intention is encoded indirectly through a design

idiom. Languages such as Rust provide an explicit module construct, illustrating an alternative in which grouping and nominal objecthood are not conflated.

2.4 Callable objects and closure surrogates

Before dedicated lambda syntax became widespread, callback behaviour in object-oriented languages was commonly encoded through objects implementing a one-method interface. Even where lambdas now exist, explicit callable classes remain common when callbacks carry state or require named types.

The semantic essence of such an entity is not primarily "object with methods" but "behaviour that can be passed as a value". Rust makes closure expressions a distinct syntactic category and assigns each closure expression a unique anonymous type. C++ similarly gives lambda expressions dedicated syntax even though their implementation is object-like.

2.5 Property-enriched nominal wrappers

A further use of types is to encode facts about values. Suppose a function accepts only supermarket-chain names that have been validated. A programmer can introduce a wrapper such as:

```
class SupermarketChainName {
    String value;
}
```

This can be an excellent use of nominal typing when the new concept is genuinely part of the domain ontology. However, the same technique is often stretched to properties such as:

```
ValidatedString
SanitizedString
NormalizedString
AuthenticatedRequest
SignatureVerifiedDocument
BoundsCheckedIndex
```

The difficulty is that independent properties compose. If n Boolean properties can be represented by nominal wrappers, as many as 2^n combinations are conceptually possible. Real code rarely defines all combinations, but the pressure toward wrapper proliferation remains.

The problem is not that such types are invalid. Rather, it is that *what a value is* and *what has been established about a value* are different forms of information.

3 Ontology and Epistemology in Program Representation

A useful conceptual distinction is between the ontology of a program value and the program's current knowledge about that value.

A conventional type judgment

$$x : T$$

primarily answers the question:

> What kind of value is x, and what operations are meaningful on it?

A static property judgment

$$x : T \, \{P_1, P_2, \ldots, P_k\}$$

answers an additional question:

> What propositions about this particular value have been established at this point in the program?

For example:

```
String [Validated, Normalized, SQLSafe]
```

need not denote a new nominal kind of string. It denotes a string together with statically established facts about the current value. This is analogous to separating a base type from a lightweight refinement layer, but the property vocabulary may remain intentionally uninterpreted and user-defined.

This distinction suggests a broader principle: nominal types should not be forced to encode every form of static information merely because the type checker is the compiler component traditionally responsible for rejection of invalid programs.

4 A Proposed Family of Semantic Declaration Categories

The following categories are illustrative rather than prescriptive. The goal is not to propose exact syntax, but to show how role-specific syntax can expose architectural heterogeneity.

4.1 Records: data composition

```
record Person {
    String name;
    DateOfBirth dob;
}
```

A record construct might imply, by default:

- value-oriented semantics,
- explicit fields,
- generated equality or structural operations,
- no hidden mutable state beyond declared components,
- restricted or absent inheritance.

Such restrictions are not merely stylistic. They make the declaration easier to reason about and allow tools to classify it accurately.

4.2 Objects: identity, state, and invariant preservation

```
object Connection {
    private Socket socket;
    private State state;

    public void send(Buffer b) { ... }
    public void close() { ... }
}
```

An object construct would emphasize identity, encapsulated mutable state, lifecycle, and invariant-preserving operations. The compiler could permit capabilities here that are deliberately unavailable to records.

4.3 Modules: grouping without object identity

```
module Geometry {
    public double area(Circle c) { ... }
    public double area(Rectangle r) { ... }
}
```

A module has a namespace and an implementation boundary but no instances. It should therefore not require private constructors, singleton idioms, or classes containing only static members. A module may also form the unit of visibility, compilation, or dependency management.

4.4 Callables and closures: behaviour as a value

```
closure Predicate<T>(T value) -> bool;
```

or, for a concrete closure:

```
let olderThan = closure(int limit) (Person p) -> bool {
    return p.age > limit;
};
```

The language can make capture semantics, immutability, lifetime, and invocation explicit rather than representing these concerns through an arbitrary class protocol.

4.5 Properties: static knowledge orthogonal to type

```
tag request Authenticated;
tag document SignatureVerified;

void publish(Document [Validated, !Confidential] d);
String [SQLSafe] escapeSQL(String input);
```

Properties live in a separate syntactic and conceptual category. They can be propagated by data-flow analysis and erased after static checking. The underlying nominal type remains unchanged.

5 Why Naming Conventions Are Not Enough

A disciplined codebase can approximate semantic categories using names such as `*Dto`, `*Service`, `*Utils`, `*Callback`, `*Factory`, or `Validated*`. This is valuable, but conventions differ from language support in four important ways.

5.1 Conventions are advisory

A compiler cannot ordinarily prevent a class named `CustomerDto` from acquiring mutable hidden state, complex lifecycle behaviour, or subtype hierarchies. A dedicated `record` construct can make some such decisions statically enforceable.

5.2 Conventions are locally variable

Naming schemes vary across teams and generations of a codebase. Language syntax is globally recognizable. A new developer may not know whether `Manager`, `Service`, `Handler`, or `Context` has a particular architectural meaning in a specific repository.

5.3 Conventions are weak inputs to tooling

An IDE or static analyzer can infer patterns heuristically, but an explicit declaration kind is a reliable semantic signal. Code browsers could group records, modules, stateful objects, callables, and property declarations separately. Search tools could answer questions such as "show all stateful abstractions in this package" without relying on naming heuristics.

5.4 Conventions do not narrow the legal design space

The most important advantage of a language feature is not cosmetic labeling but restriction. A module can be forbidden from having per-instance state because modules have no instances. A record can be forbidden from certain forms of inheritance. A closure can be required to declare or infer captures. A static property can be prohibited from affecting runtime layout. Syntax and semantics reinforce one another.

6 Codebase Legibility as a Language-Design Objective

Programming languages are often evaluated in terms of expressiveness, runtime performance, safety, or concision. Large-scale software maintenance introduces another criterion: *legibility under incomplete familiarity*.

A developer entering an unfamiliar repository rarely reads it linearly. Instead, the developer samples declarations, follows names, inspects dependency edges, and progressively constructs a mental model. The language can either reveal architectural roles immediately or force them to be reconstructed.

Consider a repository containing 5,000 class declarations. If the language presents all of them through one category, the developer must infer which declarations are data, which are stateful services, which are namespace-like utilities, which are callbacks, and which are property wrappers. If the language encodes those categories directly, the repository becomes partially self-indexing.

This suggests a useful design measure:

$$\text{semantic signal density} = \frac{\text{architecturally relevant information visible in declarations}}{\text{syntactic and conceptual overhead}}$$

A successful language feature increases semantic signal density without making the language disproportionately complex.

7 Uniformity Versus Explicitness

There is an obvious counterargument. Uniform systems have genuine virtues. The Smalltalk tradition demonstrated the conceptual power of treating everything as an object. Java's early simplicity benefited from a comparatively small number of core constructs. Excessive specialization can produce a baroque language in which programmers must learn many overlapping declaration forms.

The proposal here therefore requires a threshold test. A new syntactic category is justified only when at least several of the following hold:

1. the semantic role occurs frequently;
2. programmers already encode it through recurring idioms;
3. readers benefit materially from seeing the distinction;
4. the compiler can enforce useful role-specific invariants;
5. tools can exploit the distinction;
6. the new construct removes boilerplate or nominal clutter;
7. the concept is stable enough to deserve language-level commitment.

Records satisfy this test in several modern languages. Modules and closures commonly do as well. The argument of this paper is that lightweight static property declarations may satisfy it too, particularly when nominal wrappers would otherwise encode transient knowledge states.

8 Relationship to Existing Language Design

The proposed direction is evolutionary rather than unprecedented. Modern languages already separate semantic roles to varying degrees.

Java added record classes to express transparent carriers for immutable data more directly than ordinary classes. Rust distinguishes structs, enums, modules, traits, functions, and closure expressions, and its reference explicitly describes modules as containers for items and closures as values with unique anonymous types. C++ retains a broad class abstraction but supplements it with namespaces and dedicated lambda syntax. These developments demonstrate a recurring pattern: concepts initially encoded through class conventions may later acquire first-class syntax when their distinct role proves sufficiently important.

The lesson is not that any one existing language has found the ideal decomposition. Rather, language evolution repeatedly trades a small increase in surface syntax for clearer semantic intent and stronger static reasoning.

9 Property Tags as a Particularly Important Case

Among the categories considered here, property tags deserve special treatment because they address a different dimension of program information.

Suppose the base type is `String`. Independent static facts might include:

```
Validated
Normalized
Sanitized
SQLSafe
HTMLSafe
AuthenticatedSource
SignatureVerified
```

Encoding every useful combination nominally is unattractive:

```
ValidatedString
ValidatedNormalizedString
ValidatedNormalizedSQLSafeString
...
```

A property system instead treats the facts as a set:

```
String [Validated, Normalized, SQLSafe]
```

Function signatures can then constrain both type and established properties:

```
void executeQuery(String [SQLSafe] query);
void publish(Document [Validated, !Confidential] d);
Document [Validated] validate(Document [Parsed] d);
```

This separates the ordinary type namespace from a property namespace and allows the compiler to propagate properties through control-flow and data-flow analysis. The properties can be erased after compilation, imposing no inherent runtime cost.

This mechanism is deliberately lighter than a general refinement-type or dependent-type system. It need not permit arbitrary predicates. User-defined tags may simply be propositions whose introduction, preservation, invalidation, and requirements are declared through language constructs and function contracts.

10 Static Semantics Enabled by Role Separation

Explicit categories become most valuable when they permit distinct static rules. A language might enforce a matrix such as Table 1.

Table 1: Illustrative role-specific semantics.

	Record	Object	Module	Closure	Property
Instances	yes	yes	no	values	no
Identity	optional/no	yes	no	optional	no
Mutable hidden state	restricted	yes	module state only	captured state	no
Inheritance	restricted	optional	no	no	no
Callable	not primary	via methods	functions	intrinsic	no
Runtime layout	data	object	none	captures	erased
Primary role	data	stateful abstraction	organization	behaviour value	static knowledge

The exact rules would vary by language, but the general benefit is clear: each construct narrows the set of plausible interpretations and illegal designs.

11 Migration and Mainstream Adoption

A practical language extension must coexist with existing ecosystems. Fortunately, semantic categories can often be introduced incrementally.

A `record` can compile to an ordinary class with generated members. A `module` can compile to a namespace, static class, or package-level function container. A `closure` can lower to a compiler-generated class or environment structure. A property annotation can be erased after static analysis. A stateful `object` can compile to the language's existing object model.

Thus, the source language can become semantically richer without requiring a radically different runtime representation. This is an important distinction between *surface-language semantics* and *runtime machinery*. Multiple source-level categories may lower to the same implementation technique while still providing value to readers and compilers.

Backward compatibility can also be preserved by treating the traditional `class` as the maximally general construct. New categories would be opt-in restrictions:

> A `class` can express anything the legacy language allowed; a `record`, `module`, or `object` expresses a more specific intent and receives stronger guarantees.

This mirrors a broader pattern in language evolution: safe or intention-revealing subsets can coexist with general-purpose escape hatches.

12 Implications for Tooling

Explicit semantic categories could improve development tools even before their strongest static guarantees are implemented.

An IDE could present separate navigation views for data models, modules, stateful objects, callables, and property declarations. Dependency graphs could distinguish data dependencies from service dependencies. Architectural visualization could become less heuristic. Code review tools could flag when a record evolves into a stateful abstraction and suggest an explicit category change. Documentation generators could describe function-property contracts alongside parameter types.

For large language models and automated code-analysis systems, the same explicit semantic signals may be valuable. A system attempting to summarize an unfamiliar repository benefits when architectural roles are encoded syntactically rather than inferred from arbitrary naming conventions.

13 Risks and Design Trade-offs

The proposal introduces several risks.

First, semantic categories may overlap. A record can have methods; a module may own state; a closure is often implemented as an object; a stateful object may also be callable. The language should therefore distinguish *primary semantic role* from implementation mechanism and avoid categories whose boundaries are impossible to explain.

Second, excessive categories can fragment generic programming. If records and objects are too distinct, libraries may require duplicate abstractions. Structural interfaces, traits, concepts, or protocols can mitigate this by allowing shared behavioural constraints across declaration categories.

Third, programmers may disagree about classification. The language should reserve dedicated syntax for cases where classification provides concrete static consequences rather than merely aesthetic taxonomy.

Fourth, property systems must address aliasing, mutation, control-flow joins, concurrency, and trusted introduction of tags. These are tractable static-analysis problems but should not be hidden beneath overly simple syntax.

Finally, migration tooling matters. If a language introduces richer categories, automated refactoring should identify ordinary classes that already satisfy the stronger restrictions and offer safe conversion.

14 A General Design Principle

The argument can be summarized in two complementary principles.

Do not collapse distinct semantic roles merely because they share an implementation mechanism. A compiler may lower both a closure and a stateful object to heap-allocated structures containing fields and function pointers, but this does not imply that programmers benefit from writing both with identical source syntax.

Do not create nominal types merely to encode transient static knowledge when a separate property system is more compositional. Nominal types should describe stable domain distinctions. Static properties can describe facts that arise, disappear, or become established as values flow through a program.

Together these principles favor a language in which source constructs expose the dimensions of program meaning that developers actually reason about.

15 Conclusion

The class is one of the most productive abstractions in programming history, but its success has encouraged it to absorb roles that are not semantically identical. In large C++ and Java systems, classes may simultaneously serve as records, modules, services, closures, callbacks, policy objects, or static-property wrappers. Skilled teams can manage this heterogeneity through idioms and naming conventions, but the burden of interpretation remains with humans and heuristic tooling.

A modestly richer language can do better. By distinguishing data records, stateful objects, modules, behaviour values, and statically tracked properties, the source code itself becomes a more faithful representation of program architecture. The benefit is not merely aesthetic. Different categories permit different invariants, better diagnostics, clearer code navigation, more precise static analysis, and lower cognitive cost for developers entering unfamiliar systems.

The objective should not be maximal syntax. It should be *useful semantic explicitness*: introduce a distinct linguistic category when the distinction is common, architecturally significant, statically enforceable, and valuable to readers. Under that criterion, separating several roles currently crowded into the class abstraction appears to be a promising direction for the evolution of mainstream compiled languages.

References

[1] Oracle and OpenJDK, *Records: Changes to the Java Language Specification*, Java SE record specification. `https://docs.oracle.com/en/java/javase/14/docs/specs/records-jls.html`

[2] The Rust Project Developers, *The Rust Reference: Struct Types*. `https://doc.rust-lang.org/stable/reference/types/struct.html`

[3] The Rust Project Developers, *The Rust Reference: Modules*. `https://doc.rust-lang.org/stable/reference/items/modules.html`

[4] The Rust Project Developers, *The Rust Reference: Closure Types*. `https://doc.rust-lang.org/reference/types/closure.html`

[5] The Rust Project Developers, *The Rust Reference: Namespaces*. `https://doc.rust-lang.org/reference/names/namespaces.html`

[6] cppreference.com, *Classes*. `https://en.cppreference.com/w/cpp/language/classes`

[7] cppreference.com, *Scope: Namespace and Lambda Scope*. `https://en.cppreference.com/w/cpp/language/scope`